

LẬP TRÌNH SHADERS

MỤC TIÊU BUỔI HỌC

- **TỔNG QUAN VỀ SHADER**

- **CÁC NGÔN NGỮ LẬP TRÌNH SHADER (LOW/HIGH LEVEL SHADER)**

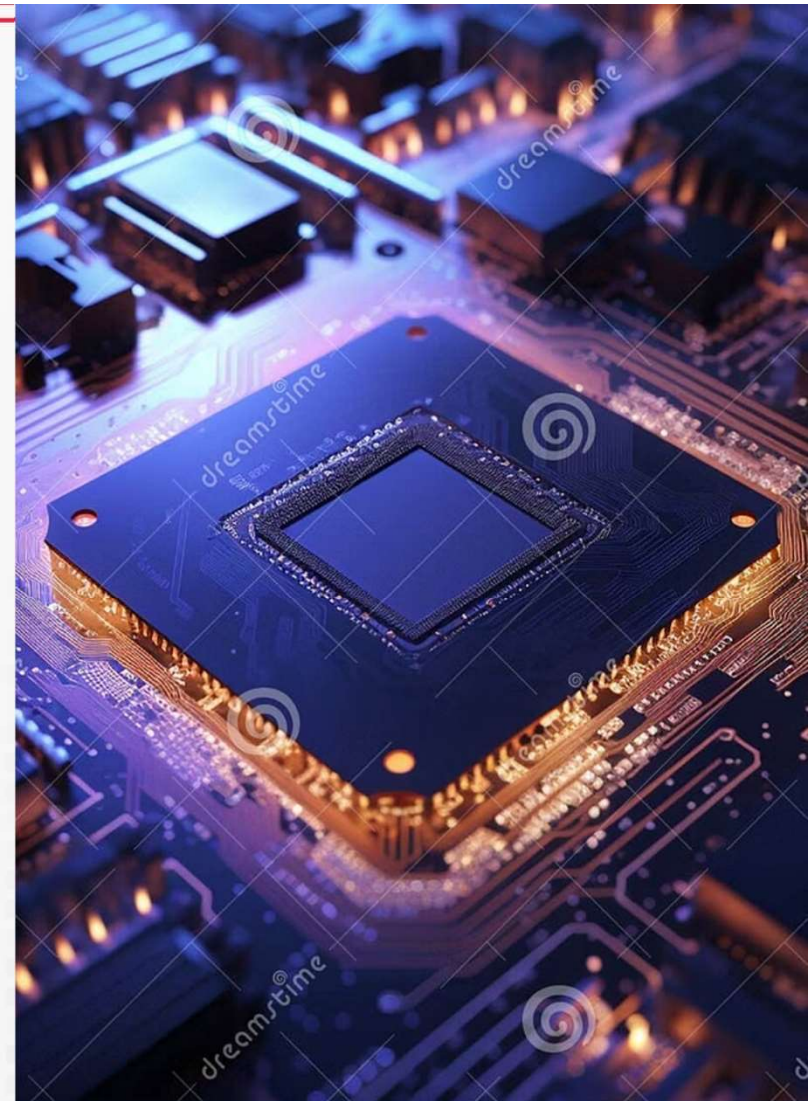
- **CÁC CÔNG NGHỆ VÀ CÔNG CỤ TRONG SHADER**

- **THỰC HÀNH**

Tổng quan về Shader

Đồ họa shader là một chương trình chuyên dụng chạy trên bộ xử lý đồ họa (GPU) của máy tính để điều chỉnh hình thức của các cảnh 3D và hình ảnh 2D.

Các chương trình này, được gọi là shader, cho phép kiểm soát chính xác quá trình kết xuất, xác định cách các đỉnh, hình học và pixel được xử lý và hiển thị trên màn hình.



Tổng quan về Shader

Trong đồ họa máy tính, các giao diện lập trình ứng dụng đồ họa (API đồ họa) được sử dụng để giao tiếp trực tiếp với phần cứng đồ họa trên máy tính hoặc máy chơi game (console).

Hai API đồ họa phổ biến nhất hiện nay là **Direct3D** và **OpenGL**.

Ứng dụng shader

1

Hiệu ứng đồ họa trong game

2

Hiệu ứng đồ họa trong phim ảnh

3

Cải thiện hiệu suất

Sức mạnh và lợi ích của Shader

Không giới hạn sáng tạo

Sự tự do và linh hoạt

Khả năng tương thích

Ưu/Nhược điểm shader

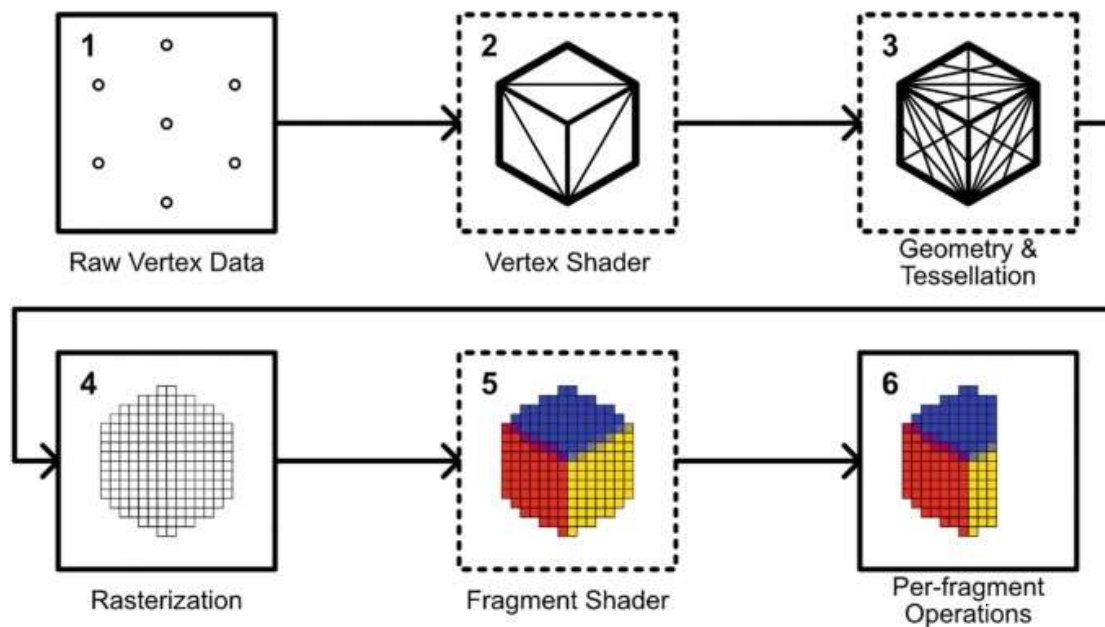
Ưu điểm

- Hiệu năng được nâng cao
- Độ chân thực hình ảnh
- Khả năng tùy chỉnh
- Tính tương tác thời gian thực

Nhược điểm

- Hạn chế về số lượng tính năng và cách các nhà phát triển giao tiếp với phần cứng
- Thiếu khả năng tùy chỉnh cho phép các nhà phát triển tự viết thuật toán và thực thi chúng trên phần cứng đồ họa

Vị trí Shader trong Render Pipeline



Phân loại Shader theo chức năng

Hiện nay có ba loại shader khác nhau được sử dụng để xử lý các dữ liệu cấu thành nên một môi trường ảo:

→ **Vertex shaders (bộ đồ bóng đỉnh)** → **Geometry shaders (bộ đồ bóng hình học)** → **Pixel shaders (bộ đồ bóng điểm ảnh)**

❏ **Chương trình Shader (Shader program):** Khi kết hợp lại với nhau trong cùng một hiệu ứng, tập hợp các shader này được gọi chung là một chương trình shader.

❏ **Quy tắc hoạt động:** Tại một thời điểm, chỉ có một loại shader được kích hoạt (ví dụ: không thể chạy hai vertex shader cùng lúc trên cùng một dữ liệu).

Vertex Shaders (Bộ đồ bóng đỉnh)

Khái niệm

Là mã thực thi trên từng đỉnh (vertex) được truyền vào phần cứng kết xuất đồ họa.

Nguồn dữ liệu

Dữ liệu đầu vào của vertex shader đến trực tiếp từ chính ứng dụng.

Chức năng chính

Thường được sử dụng để biến đổi vị trí các đỉnh bằng các ma trận (như ma trận model-view projection) và thực hiện các tính toán trên từng đỉnh.

Các phép toán phổ biến

Chiếu sáng ở cấp độ đỉnh (per-vertex lighting), hiệu ứng hoạt hình bằng GPU (GPU animations), dịch chuyển đỉnh (vertex displacements), và tính toán các giá trị để nội suy qua bề mặt cho pixel shader xử lý.

Geometry Shaders (Bộ đồ bóng hình học)



Vị trí trong quy trình

Nằm ở giữa Vertex shader và Pixel shader.



Nguồn dữ liệu

Nhận dữ liệu đầu vào đã qua xử lý từ Vertex shader.



Chức năng chính

Hoạt động trên toàn bộ các khối hình học nguyên thủy (primitives). Nó có khả năng tạo ra (emit) các hình học mới hoặc loại bỏ các hình học ban đầu.



Tính tương thích

Đây là một loại shader mới có mặt trong Shader Model 4.0, được hỗ trợ bởi các API đồ họa như Direct3D 10 và OpenGL 3.0 trở lên.

Pixel Shaders (Bộ đồ bóng điểm ảnh)

Tên gọi khác

Còn được biết đến với tên gọi là Fragment shader.

Khái niệm

Hoạt động trên từng điểm ảnh (pixel) đã được raster hóa (màn hình hóa) để hiển thị lên màn hình.

Nhắc lại về Raster hoá

chúng ta đã thấy rằng một khi một hình khối cơ bản đã được biến đổi và cắt xén, vùng tạo nên hình khối đó sẽ được tô bóng. Nếu bạn đang sử dụng pixel shader, mỗi pixel được tô bóng và nằm trong vùng của hình khối cơ bản sẽ được xử lý.

Chức năng chính

Xử lý và thực thi thuật toán trên các điểm ảnh nằm bên trong khu vực của khối hình học nguyên thủy.

Nguồn dữ liệu

Đầu vào của pixel shader có thể là đầu ra từ vertex shader hoặc từ geometry shader (nếu có sử dụng).

PHÂN LOẠI SHADER THEO NGÔN NGỮ LẬP TRÌNH



LOW-LEVEL SHADERS

- **Lịch sử phát triển**

Trong những ngày đầu của shader lập trình được, các nhà phát triển phải sử dụng hợp ngữ (assembly language) theo kiến trúc RISC (Reduced Instruction Set Compiler) để lập trình các hiệu ứng đồ họa. Đây là một thiết kế CPU ưu tiên sử dụng tập lệnh có kích thước và độ phức tạp được giảm thiểu.

- **Khó khăn khi sử dụng**

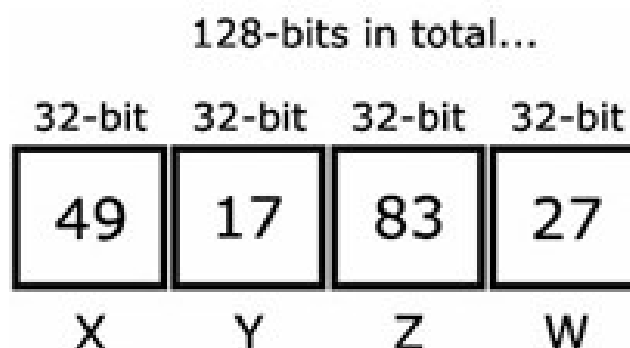
Việc viết mã, đọc và bảo trì bằng hợp ngữ khó khăn hơn rất nhiều so với các ngôn ngữ bậc cao (như C/C++).

- **Sự phân mảnh của API**

Các API đồ họa chính (như OpenGL và Direct3D) có ngôn ngữ assembly khác nhau.

Cấu trúc Dữ liệu và Xử lý Cơ bản

- Kiểu dữ liệu: Mọi dữ liệu trong shader được lưu trữ trong các biến dấu phẩy động (floating-point) 128-bit, có cấu trúc tương tự như một vector 4 chiều (4D vectors).
- Xử lý đa luồng (SIMD): Phần cứng đồ họa sử dụng kiến trúc bộ xử lý SIMD (Single Instruction Multiple Data - Một lệnh nhiều dữ liệu). Điều này cho phép một tập lệnh duy nhất thao tác đồng thời trên nhiều phần tử của vector cùng lúc (ví dụ: thực hiện phép toán trên cả 4 thành phần x, y, z, w trong một lệnh).



Những hạn chế kỹ thuật đời đầu

Thiếu cấu trúc điều khiển

Trong giai đoạn đầu, shader bậc thấp không hỗ trợ vòng lặp (loops) hay câu lệnh điều kiện (if/else).

Giới hạn số lượng tập lệnh

Shader chịu sự giới hạn nghiêm ngặt về độ dài của mã nguồn

Hiệu suất phép toán

Các thao tác như chuẩn hóa (normalization), căn bậc hai hay lũy thừa từng rất chậm, buộc lập trình viên phải sử dụng các thủ thuật như "kết cấu tra cứu" (lookup textures) để tăng tốc độ.

Hệ thống Thanh ghi (Registers) trong Vertex Shader

Lập trình assembly chủ yếu xoay quanh việc phân bổ và sử dụng các thanh ghi. Có 4 loại thanh ghi chính:

Thanh ghi đầu vào

Input registers - v0 đến v15

Lưu trữ thông tin đầu vào (như tọa độ, màu sắc của đỉnh) từ ứng dụng truyền vào.

Thanh ghi đầu ra

Output registers

Có khoảng 13 thanh ghi dùng để lưu thông tin sau khi xử lý xong để chuyển sang giai đoạn tiếp theo (chuyển cho pixel shader).

Thanh ghi tạm thời

Temporary registers - r0 đến r11

Dùng để lưu trữ tạm thời các giá trị trung gian trong quá trình tính toán thuật toán.

Thanh ghi hằng số

Constant registers

Chứa các dữ liệu không thay đổi giữa các đỉnh hoặc các điểm ảnh (như ma trận model-view projection). Shader chỉ có quyền đọc các thanh ghi này chứ không thể ghi đè lên chúng.

Khác biệt giữa Vertex và Pixel Shader bậc thấp

Vertex Shader

Thường có số lượng thanh ghi hằng số lớn hơn (có thể lên tới 96 hoặc 192 tùy phần cứng).

Pixel Shader

Bị hạn chế hơn nhiều về tài nguyên trong những phiên bản đầu tiên. Ví dụ, Shader Model 1.1 đến 1.3 chỉ có 8 thanh ghi hằng số và 2 thanh ghi tạm thời

Quy trình làm việc với Low-Level Shaders

Biên dịch

Shaders cấp thấp có thể được biên dịch sẵn dưới dạng tệp nhị phân (binary) hoặc tải dưới dạng tệp văn bản ASCII và biên dịch trực tiếp khi ứng dụng đang chạy (run-time).

Các bước thiết lập và sử dụng

Để nạp và kết xuất bằng một low-level shader trong OpenGL hoặc Direct3D, ứng dụng cần trải qua các bước chung sau:

1

Tải vertex shader và pixel shader vào bộ nhớ

2

Kích hoạt (Bind) vertex shader và pixel shader cho quy trình kết xuất

3

Gán giá trị cho các biến hằng số (thanh ghi) mà shader cần dùng

4

Liên kết các textures (hình ảnh) và thiết lập các trạng thái kết xuất cần thiết cho hiệu ứng

5

Vẽ/kết xuất hình học (Render the geometry)

Giới thiệu về High-Level Shaders

1

Sự thay thế hoàn hảo

Ngôn ngữ đồ bóng cấp cao (High-level shading languages) mang đến một giải pháp linh hoạt để thay thế cho các ngôn ngữ đồ bóng cấp thấp (assembly) từng được sử dụng phổ biến trong giai đoạn đầu của phần cứng đồ họa lập trình được.

2

Cú pháp quen thuộc

Hầu hết các ngôn ngữ đồ bóng cấp cao đều được xây dựng dựa trên cú pháp của ngôn ngữ C/C++, giúp các lập trình viên đã quen thuộc với ngôn ngữ này dễ dàng tiếp cận.

Ưu điểm so với Shader bậc thấp

So với các ngôn ngữ cấp thấp, ngôn ngữ cấp cao mang lại 5 lợi ích vượt trội:



Dễ đọc hơn

Easier to read



Dễ chỉnh sửa và bảo trì

Easier to modify



Dễ dàng theo dõi và dò lỗi

Easier to track errors



Dễ viết mã

Easier to write



Thời gian phát triển nhanh hơn

Faster to develop: Lập trình bằng shader cấp thấp thường mất nhiều thời gian hơn so với các lựa chọn cấp cao

Nhược điểm và Sự đánh đổi

Dù có nhiều lợi ích, việc dùng ngôn ngữ cấp cao cũng đòi hỏi sự đánh đổi:

Mất quyền kiểm soát phần cứng

Lập trình viên không còn quyền kiểm soát tối đa đối với cách phân bố và sử dụng các thanh ghi (registers) của phần cứng.

Phụ thuộc vào trình biên dịch (Compiler)

Trình biên dịch đảm nhận việc dịch mã cấp cao sang mã cấp thấp. Dù các trình biên dịch hiện nay rất thông minh, đôi khi chúng vẫn không tạo ra được mã nguồn nhanh nhất và tối ưu nhất.

- ❑ **Kết luận:** Dù mất đi một phần quyền kiểm soát cấp thấp, sự tiện lợi của ngôn ngữ cấp cao hầu hết đều xứng đáng với sự đánh đổi này.

Biến và Kiểu dữ liệu (Variables & Data Types)

Kiểu dữ liệu

Các biến được khai báo bằng các kiểu dữ liệu tích hợp sẵn như số nguyên (integer) hoặc dấu phẩy động (floating-point).

Xử lý ẩn

Trình biên dịch sẽ tự động phiên dịch các biến này thành các thanh ghi phần cứng tương ứng (thanh ghi đầu vào, đầu ra, hằng số hoặc tạm thời) trên GPU.

Sự khác biệt với cấp thấp

Số nguyên là một tính năng của ngôn ngữ cấp cao chứ không phải là loại thanh ghi phần cứng.

Phạm vi của Biến (Variable Scopes)

Trong High-level shaders, các biến có thể được xác định với 3 cấp độ phạm vi (scope):

Phạm vi hàm (Function scope)

Biến chỉ có tác dụng và hiển thị cục bộ bên trong hàm chứa nó.



Phạm vi Shader (Shader scope)

Hoạt động như biến toàn cục (global variable) bên trong một shader, có thể được nhìn thấy bởi bất kỳ khối mã hoặc hàm nào trong shader đó.

Phạm vi chương trình (Program scope)

Biến toàn cục cấp cao nhất, có thể được nhìn thấy bởi tất cả các shader (vertex, pixel, geometry) cấu thành nên toàn bộ chương trình shader (shader program).

Hàm (Functions)

Cấu trúc tương tự C++

Các hàm trong shader cấp cao hoạt động giống như trong ngôn ngữ C++

Tính năng hỗ trợ

Hỗ trợ định nghĩa kiểu dữ liệu trả về (return types), tham số đầu vào (parameters), và các biến cục bộ (local variables) theo quy tắc phạm vi độc lập

Xử lý tham chiếu

Không hỗ trợ các toán tử hủy tham chiếu (dereferencing operators / con trỏ) trực tiếp như C/C++, nhưng ngôn ngữ cung cấp các từ khóa đặc biệt (ví dụ: in, out, inout) để mô phỏng chức năng truyền tham chiếu

Các ngôn ngữ High-Level Shader phổ biến

- **NVIDIA Cg (C for graphics):** Ngôn ngữ đa nền tảng do NVIDIA tiên phong phát triển.
- **Microsoft HLSL (High-Level Shading Language):** Ngôn ngữ tiêu chuẩn của Direct3D, có sự tương đồng lớn với Cg do Microsoft từng hợp tác cùng NVIDIA.
- **OpenGL GLSL (OpenGL Shading Language):** Ngôn ngữ đồ bóng tiêu chuẩn dùng cho OpenGL. Tất cả đều dựa trên ngôn ngữ C và chia sẻ cấu trúc rất giống nhau.

Tổng quan về HLSL (High-Level Shading Language)

Định nghĩa

HLSL là ngôn ngữ đồ bóng cấp cao tiêu chuẩn được Microsoft tạo ra dành cho API đồ họa Direct3D và khung phát triển XNA.

Nguồn gốc

Nó chia sẻ nhiều điểm tương đồng với ngôn ngữ Cg do Microsoft từng hợp tác phát triển với NVIDIA.

Cú pháp

Dựa trên C, hỗ trợ các chỉ thị tiền biên dịch (preprocessor directives), cấu trúc điều khiển luồng linh hoạt và bộ hàm tích hợp (built-in functions) cực kì phong phú.

Sự khác biệt về từ khóa

Khác với GLSL, vectơ 4 chiều trong HLSL là **float4** (thay vì vec4), và ma trận 4x4 là **float4x4** (thay vì mat4).

Tổng quan về GLSL (OpenGL Shading Language)



Định nghĩa

GLSL là ngôn ngữ đồ bóng cấp cao tiêu chuẩn đi kèm với API đồ họa OpenGL.



Đặc điểm

GLSL được xây dựng dựa trên cú pháp của ngôn ngữ C/C++.



Cấu trúc cơ bản

Điểm bắt đầu của một shader trong GLSL luôn là hàm main(). Hàm này bao hàm thuật toán của shader và không trả về giá trị nào (kiểu void).



Độ an toàn kiểu dữ liệu (Type safe)

Khác với C++, GLSL không hỗ trợ ép kiểu ngầm định (implicit type casting). Nghĩa là các kiểu dữ liệu phải hoàn toàn khớp nhau khi gán biến, hoặc lập trình viên phải thực hiện ép kiểu tường minh

NVIDIA's Cg

Cg (viết tắt của C for graphics) là một ngôn ngữ đồ bóng bậc cao được NVIDIA phát triển tại thời điểm các shader bắt đầu trở nên phổ biến trong ngành công nghiệp đồ họa.

Tính đa nền tảng

Điểm mạnh lớn nhất của Cg là khả năng chạy trên nhiều nền tảng và API đồ họa khác nhau. Nó được hỗ trợ trên:

- **Hệ điều hành:** Windows (32 và 64-bit), Mac, Linux và Solaris.
- **API đồ họa:** Cả OpenGL và Direct3D.
- **Phần cứng:** Bao gồm cả máy chơi game PlayStation 3.

Lợi ích

Đối với các nhà phát triển đã quen thuộc với HLSL, Cg sẽ mang lại cảm giác rất gần gũi vì hai ngôn ngữ này cực kỳ tương đồng. Cg cho phép lập trình viên chỉ cần viết một bộ shader duy nhất nhưng có thể sử dụng được trên nhiều hệ thống khác nhau thay vì phải viết riêng biệt cho từng API.

Các công nghệ và công cụ khác

Pixar's RenderMan

RenderMan là một bộ sưu tập các công cụ có khả năng tùy biến cao được tạo ra bởi Pixar, một trong những công ty hàng đầu thế giới về hoạt hình và hiệu ứng kỹ thuật số.

Các thành phần chính

Bộ công cụ này bao gồm RenderMan for Maya, RenderMan Studio và RenderMan Pro Server.

Các tính năng cung cấp cho nghệ sĩ

- Hỗ trợ kết xuất qua mạng với các hệ thống render farm (trạm kết xuất đồ họa).
- Hệ thống kết xuất (rendering system) mạnh mẽ.
- Quản lý và tạo ngôn ngữ đồ bóng bậc cao.
- Công cụ xử lý hình ảnh chuyên sâu.

Ứng dụng

RenderMan thường được tích hợp vào phần mềm Maya Pro và được sử dụng rộng rãi trong các bộ phim hoạt hình chất lượng cao như The Incredibles.

NVIDIA's FX Composer

Chức năng

Đây là một công cụ miễn phí giúp việc tạo shader trở nên dễ dàng và năng suất hơn.

Tiện ích

Nó cung cấp tính năng xem trực tiếp (live preview) và kiểm tra cú pháp để phát hiện lỗi ngay trong lúc viết mã thay vì phải đợi đến lúc chạy ứng dụng mới phát hiện được lỗi.